

GUÍA DE LA ENTREGA 2 — LÉELO COMPLETO, PASO A PASO

Vence: domingo **20-sep-2026, 23:59** (hora Argentina). **Vale:** 7 % de la nota final. **Tag en GitHub:** exactamente `entrega-2`.

Esta guía repite lo que ya está en `consigna.md` y `rubrica.md`, pero paso a paso y sin vueltas. La Entrega 2 es **incremental**: conserva todo lo que anduvo en E1 y le suma 4 cosas nuevas.

1. LO PRIMERO: las 4 cosas que se corrigen esta vez

Son 4 ítems de la rúbrica. Si hacés solo esto, aprobás la E2:

#	Qué se corrige	Peso	En palabras simples
1	Código partido en módulos ; clases del dominio; <code>main</code> no concentra la lógica	20 %	Tu código NO puede ser un solo archivo de 800 líneas con todo adentro.
2	Recursión del dominio con caso base, sobre tus datos reales	35 %	No es un factorial. Es recursión de TU tema (Pokédex, recetas o música).
3	Traza de un ejemplo de esa recursión en el informe	15 %	Mostrar, llamada por llamada, cómo se resuelve un caso real.
4	Protocolo de pruebas con al menos 8 casos escritos	20 %	Una tabla con 8 pruebas: qué hacés, qué esperás. No hace falta que estén corridas.
5	Declaración de IA actualizada + que el tag corre	10 %	Llenar la fila E2 y que <code>entrega-2</code> funcione.

Regla de oro: si algo que andaba en E1 dejó de andar, E2 se puntúa 0 en ese ítem nuevo aunque todo lo nuevo esté perfecto. **No rompas lo que ya funcionaba.**

2. QUÉ TENÉS QUE TENER ANTES DE EMPEZAR

- ☐ [] Tu repo con el tag `entrega-1` andando (si E1 salió bien, no lo toques “de más”).
- ☐ [] Tu tema elegido en `src/config.py` (ya lo tenías de E1).
- ☐ [] El esqueleto con esta estructura **sin borrar**:

```

src/
  main.py          <- acá SOLO el menú y el despacho de opciones
  config.py
  excepciones.py
  dominio/         <- TUS clases del tema (Pokémon, Receta, Canción, etc.)
  tads/            <- se completa en E3, NO lo toques todavía
  algoritmos/      <- se completa en E4, NO lo toques todavía
  persistencia/    <- se completa en E5, NO lo toques todavía
docs/
  INFORME.md       <- va a crecer cada entrega
  PROTOCOLO_PRUEBAS.md
  DECLARACION_IA.md
data/
  *.txt            <- NO se leen hasta E5. No los borres ni los vacíes.

```

3. ÍTEM 1 (20 %): MÓDULOS Y CLASES DEL DOMINIO

3.1 Qué se pide

Tu código tiene que estar **repartido en archivos (módulos) con responsabilidades claras**. Cada concepto de tu tema va en su propio archivo dentro de `src/dominio/`.

3.2 Qué clases armar según tu tema

Tema	Clases que necesitás por lo menos	Archivo sugerido
Pokédex	Pokemon , Pokedex (catálogo), Equipo	src/dominio/pokemon.py , src/dominio/pokedex.py
Recetario	Receta , Recetario (libro), MenuSemanal	src/dominio/receta.py , src/dominio/recetario.py
Música	Cancion , Biblioteca , Playlist	src/dominio/cancion.py , src/dominio/biblioteca.py

Pueden usar una `list` de Python adentro por ahora (eso se cambia en E3). Lo importante en E2 es:

- Las clases **existen**, están separadas y tienen métodos.
- `main.py` **solo** muestra el menú y llama a funciones/métodos del dominio. No puede contener la lógica del negocio.

3.3 Ejemplo de cómo NO quedaría

```
# main.py – FORMA INCORRECTA (todo junto)
pokemones = []
def cargar():
    pokemones.append({"nombre": "Bulbasaur", "hp": 45})
def listar():
    for p in pokemones:
        print(p["nombre"])
```

```
# src/dominio/pokemon.py – FORMA CORRECTA
class Pokemon:
    def __init__(self, nombre, hp, ataque):
        self.nombre = nombre
        self.hp = hp
        self.ataque = ataque

    def resumen(self):
        return f"{self.nombre} (hp {self.hp})"
```

```
# main.py – FORMA CORRECTA (solo despacha)
from src.dominio.pokedex import Pokedex

def main():
    dex = Pokedex()
    dex.cargar_datos_iniciales()
    # ... menú que llama a dex.listar(), dex.buscar(), etc.
```

Checklist módulos:

- ☐ [] Hay por lo menos 1 archivo nuevo en `src/dominio/` con clases.
- ☐ [] `main.py` no supera lo necesario para el menú y la llamada.
- ☐ [] Cada clase tiene un **único** trabajo claro.

4. ÍTEM 2 (35 %): LA RECURSIÓN DEL DOMINIO — LA PARTE MÁS IMPORTANTE

4.1 Qué se pide

Una función **recursiva de verdad** (se llama a sí misma), que tenga:

- **Caso base:** cuándo se corta la recursión y se devuelve algo.
- **Caso recursivo:** se llama a sí misma con un problema más chico.

Y que opere **sobre los datos de tu tema**. Un `factorial`, `fibonacci` o `contar del 1 al n` vale **0** en este ítem: es de juguete, no es del dominio.

4.2 Qué función hacer según tu tema (usá ésta de referencia)

Tema	Función recursiva	Sobre qué datos
Pokédex	Mostrar la cadena de evoluciones hacia adelante (y si pueden, PRE-evoluciones hacia atrás)	<code>data/evoluciones.txt</code> (1→2→3, 172→25→26, 133→134/135/136...)
Recetario	Desglosar una receta en todos los sub-ingredientes , explotando las sub-recetas	<code>data/subrecetas.txt</code> + <code>data/ingredientes.txt</code>
Música	Listar todas las versiones derivadas de una canción (cover, live, remix), y las versiones de las versiones	<code>data/versiones.txt</code> (13 es live de 12, 62 es live de 1...)

4.3 Ejemplo concreto y real — Pokédex

Dato del `evoluciones.txt` : 172,25 y 25,26 . Es decir: **Pichu (172) → Pikachu (25) → Raichu (26)**.

```
# cadena de evoluciones hacia adelante
def cadena_evolucion(pokedex, id_pokemon, visto_vacio=False):
    pokemon = pokedex.buscar(id_pokemon)          # si no existe: caso base
    if pokemon is None:
        return []
    siguiente = pokedex.siguiente_evolucion(id_pokemon) # None si no evoluciona
    if siguiente is None:                             # CASO BASE: última evolución
        return [id_pokemon]
    return [id_pokemon] + cadena_evolucion(pokedex, siguiente) # CASO RECURSIVO
```

Salida para `Pichu` : [172, 25, 26] → Pichu → Pikachu → Raichu.

4.4 Ejemplo concreto y real — Recetario

Dato del `subrecetas.txt` : 10,3 y 10,5 . Es decir, **Empanadas de carne (10)** → usa el Sofrito (3) y la Masa de empanadas (5). Y `recetas.txt` : el Sofrito (3) no tiene sub-recetas.

```
def desglosar_subrecetas(recetario, id_receta):
    sub = recetario.subrecetas(id_receta)          # Lista de ids
    if not sub:                                    # CASO BASE: no tiene sub-recetas
        return [id_receta]                         # es una receta "hoja"
    resultado = [id_receta]
    for s in sub:                                  # CASO RECURSIVO
        resultado += desglosar_subrecetas(recetario, s)
    return resultado
```

Salida para `10`: `[10, 3, 5]`. La receta "Empanadas" se compone del Sofrito y la Masa (recetas hoja).

4.5 Ejemplo concreto y real — Música

Dato del `versiones.txt`: `13,12,live` (la canción 13 es un live de la 12). Si la 12 tuviera otras derivadas, se listan también → ahí está la recursión.

```
def versiones_de(versionable, id_cancion):
    directas = versionable.versiones_directas(id_cancion) # covers/lives/remixes
    if not directas:                                     # CASO BASE
        return []
    resultado = list(directas)
    for v in directas:                                  # CASO RECURSIVO
        resultado += versiones_de(versionable, v)
    return resultado
```

4.6 Checklist recursión

- ☐ [] La función se llama a sí misma.
- ☐ [] Tiene caso base explícito y se puede explicar cuál es.
- ☐ [] Usa datos reales de tu tema (ids, nombres, títulos). No es un factorial.
- ☐ [] La probás con un ítem **con** cadena y otro **sin** cadena (para ver el caso base).

5. ÍTEM 3 (15 %): LA TRAZA EN EL INFORME

5.1 Qué se pide

En `docs/INFORME.md`, en la columna "Recursión (E2)", mostrar **paso a paso** cómo se resuelve la recursión para **un caso real** de tus datos. Es como "mostrar las cartas sobre la mesa": qué pasa en cada llamada, hasta llegar al caso base, y cómo se "desapila".

5.2 Plantilla para escribirla (ejemplo con la Pokédex y `cadena_evolucion(Pichu)`)

Función: `cadena_evolucion(pokedex, id)`

Caso base: si el Pokémon no tiene evolución siguiente → devolver `[id]`.

Caso recursivo: `[id] + cadena_evolucion(pokedex, siguiente)`

Traza para Pichu (id 172): según `evoluciones.txt`, 172→25→26.

Llamada 1: `cadena_evolucion(172)` → tiene siguiente (25)
→ devuelve `[172] + cadena_evolucion(25)`

Llamada 2: `cadena_evolucion(25)` → tiene siguiente (26)
→ devuelve `[25] + cadena_evolucion(26)`

Llamada 3: `cadena_evolucion(26)` → NO tiene siguiente (caso base)
→ devuelve `[26]`

Resultado: `[172] + [25] + [26] = [172, 25, 26]`

- ☐ [] La traza usa datos reales de tu dataset.
- ☐ [] Muestra cada llamada recursiva y el caso base.
- ☐ [] Muestra el resultado final.

6. ÍTEM 4 (20 %): PROTOCOLO DE PRUEBAS CON 8 CASOS

6.1 Qué se pide

En `docs/PROTOCOLO_PRUEBAS.md` completar **al menos 8 filas** de la tabla. Cada fila es un caso de prueba **escrito** (todavía **no hace falta** que estén ejecutados ni que pasen; eso es de E3).

6.2 Ya tenés 4 casos listos (P01–P04)

El esqueleto ya trae 8 filas de ejemplo. Las de E1 y E2 son:

ID	Qué probar	Resultado esperado
P01	Arrancar el programa y listar catálogo	lista no vacía, sin error
P02	Buscar un ítem inexistente (id = -1)	mensaje claro, el menú sigue
P03	Recursión sobre un ítem CON cadena	imprime la cadena completa
P04	Recursión sobre un ítem SIN derivados	solo el propio ítem (caso base)

Necesitás 4 más. ¿Por cuáles puedo arrancar? Cualquier prueba manual de E1 o de esta entrega, por ejemplo:

ID	Acción	Resultado esperado
P05	Ver el detalle de un ítem que existe	muestra todos sus datos
P06	Ver el detalle de un ítem que NO existe	mensaje claro, no se corta el programa
P07	Elegir una opción de menú inválida (ej. "9z")	vuelve a mostrar el menú
P08	Pasar enter vacío en el menú	no explota; vuelve a preguntar

Escribí la columna **Resultado** como `no corrido` (todavía).

- ☐ [] La tabla tiene 8 o más filas.
- ☐ [] Cada fila tiene: Acción (pasos), Datos, Resultado esperado.
- ☐ [] Están redactadas para que otra persona pueda ejecutarlas sin que le expliques nada.

7. ÍTEM 5 (10 %): DECLARACIÓN DE IA Y TAG

- ☐ [] `docs/DECLARACION_IA.md` : completar la fila **E2** (fecha, herramienta, para qué, qué hicieron, quién). Si no usaste IA, igual poné una fila que diga "no usamos IA en esta entrega". **No declarar IA cuando sí se usó anula la entrega.**
- ☐ [] El tag `entrega-2` existe y el programa **arranca**.

8. PASO A PASO PARA ENTREGAR (copiá y pegá)

Desde la raíz de tu repo:

```
git add .
git status                # revisá que esté todo lo que querés subir
git commit -m "E2: módulos, clases y recursión del tema"
git tag entrega-2
git push
git push origin entrega-2
```

Si te equivocaste **antes** del domingo 23:59 podés borrar y recrear el tag (está en `GIT.md`). **Después** de la hora, no: ese tag ya se está corrigiendo.

Luego mandá el **mail de la entrega** a los dos docentes:

Destinatarios	diego.ambrossio@unab.edu.ar y angel.bianco@unab.edu.ar (en el mismo correo)
Asunto	AyED C2 2026 Comision X - GRUPO XX - Entrega 2

Destinatarios	diego.ambrossio@unab.edu.ar y angel.bianco@unab.edu.ar (en el mismo correo)
Cuerpo	Repo: <code>https://github.com/<usuario>/<repo></code> + Entrega de E2: <1 línea con lo que incluye el tag>

Ejemplo de cuerpo: Repo: `https://github.com/usuario/ayed-2026-recetario` y Entrega de E2: código en módulos, clases del dominio, recursión del tema con traza en el informe y protocolo de pruebas con 8 casos escritos .

- ☐ [] En el campus/GitHub: cargar la URL del repo y el tag `entrega-2` .
- ☐ [] Sin el mail **no se corrige la entrega**. Es obligatorio, como el de E1.

9. CHECKLIST FINAL ANTES DEL DOMINGO 23:59

Marcá todo esto antes de tagear:

- ☐ [] `src/dominio/` tiene tus clases del tema y no tocaste `tads/` , `algoritmos/` , `persistencia/` (eso es de E3, E4 y E5).
- ☐ [] `main.py` es solo menú + llamadas.
- ☐ [] La recursión es de tu dominio, con caso base, y la probaste con un ítem con cadena y otro sin cadena.
- ☐ [] `docs/INFORME.md` tiene la traza real.
- ☐ [] `docs/PROTOCOLO_PRUEBAS.md` tiene 8+ casos escritos.
- ☐ [] `docs/DECLARACION_IA.md` tiene la fila E2.
- ☐ [] Lo de E1 (listar catálogo, ver detalle, etc.) sigue andando.
- ☐ [] `python -m src.main` arranca sin error.
- ☐ [] `git tag entrega-2` hecho, pusheado, y **también** `git push origin entrega-2` .
- ☐ [] Mail a Ambrossio y Bianco con asunto `...Entrega 2` , repo y 1 línea de qué incluye.

10. ERRORES QUE CUESTAN PUNTOS (y cómo evitarlos)

Error	Consecuencia según rúbrica	Cómo evitarlo
Un solo archivo gigante con todo	Ítem 2.1 en N	Partir en <code>src/dominio/*.py</code> .
Factorial / fibonacci como "recursión"	Ítem 2.2 en N	La recursión tiene que usar datos de tu tema.
Recursión que explota con ítem sin cadena	Ítem 2.2 en P	Probar el caso base.

Error	Consecuencia según rúbrica	Cómo evitarlo
Sin traza en el informe	Ítem 2.3 en N	Copiar la plantilla de la sección 5.
Menos de 8 casos de prueba	Ítem 2.4 en P o N	Completar la tabla hasta 8 como mínimo.
Declaración de IA sin actualizar	Ítem 2.5 en N	Llenar la fila E2.
Tag con otro nombre (entrega2 , E2 , etc.)	No se encuentra la entrega	Exactamente entrega-2 .
Clonás el repo pero no corre	0 en “funciona”	Probar <code>python -m src.main</code> desde la raíz.
Entregás después del domingo 23:59	–25 % si llega el lunes o martes 23:59; después, no se acepta	Terminar antes. La próxima entrega (E3) cierra 10 días antes del parcial.